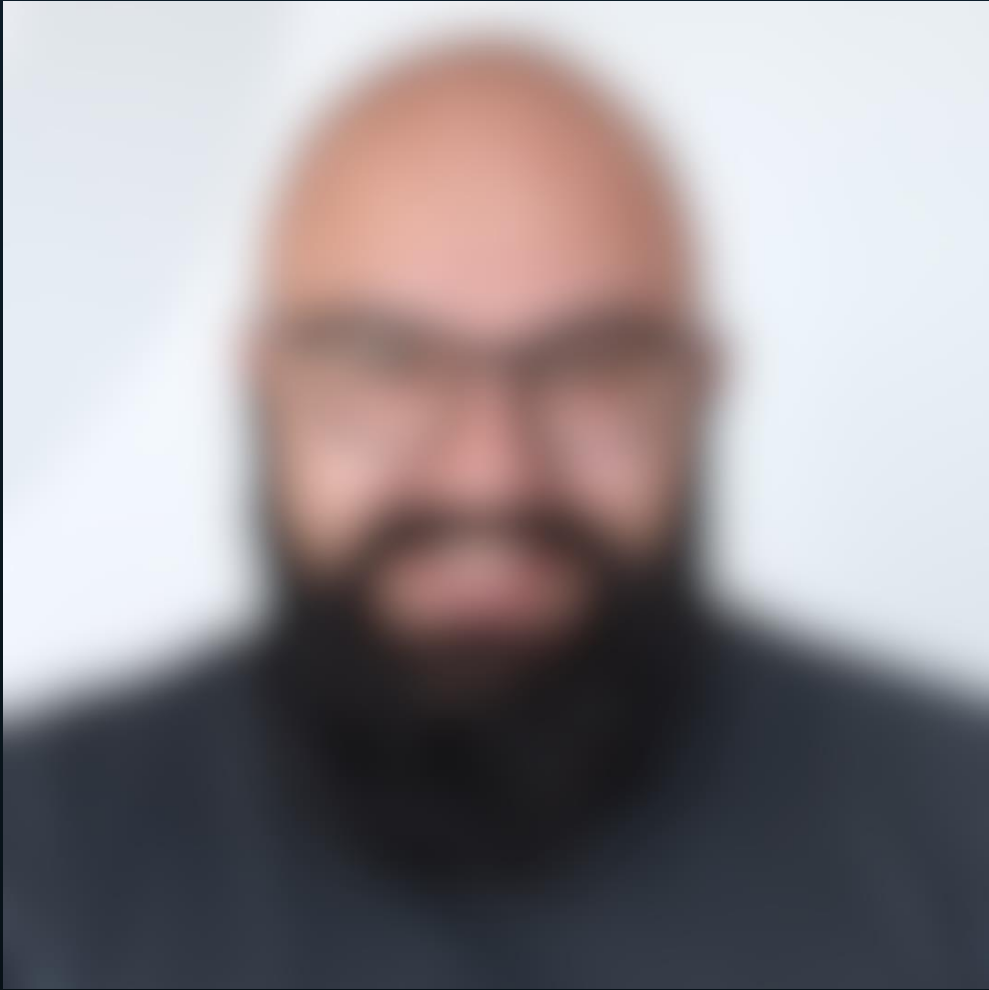


FROM RACE CONDITIONS TO WORKER POOLS: MASTERING GO CONCURRENCY

Salvatore Caputi
Working Software Conference 2026



WHO AM I?



WHO AM I?



Solutions Architect
@Pixartprinting



Kenjutsu practitioner
@Bushido Dojo Venezia



Bojutsu practitioner
@Bushido Dojo Venezia



Iaido practitioner
@Bushido Dojo Venezia



AGENDA

01. Goal

02. Why Go

03. What is concurrency

04. Goroutine

05. Mutex

06. Wait Group

07. Channel

08. Worker Pool

09. Select (extra topic)

10. Q&A

GOALS

Build scripts together

Improve previous code by adding new «pieces»

Use simple primitives, no complex external libraries

Get familiar with Go and the concurrency model

NON-GOALS

Deep dive into internal scheduler or runtime mechanism

Usage of non-primitive solutions

WHY GO

Safe by design

Concurrency primitives **built-in**

No need to deal with **processes** and **threads**

One codebase, any **platform**

WHAT IS CONCURRENCY

Concurrent programming is a programming paradigm that allows multiple tasks to be executed simultaneously, sharing resources and interacting with each other.

It is a way of writing code that allows multiple tasks to be executed at the same time, sharing resources and interacting with each other.

Concurrent programming is a programming paradigm that allows multiple tasks to be executed simultaneously, sharing resources and interacting with each other.

It is a way of writing code that allows multiple tasks to be executed at the same time, sharing resources and interacting with each other.

Concurrent programming is a programming paradigm that allows multiple tasks to be executed simultaneously, sharing resources and interacting with each other.

It is a way of writing code that allows multiple tasks to be executed at the same time, sharing resources and interacting with each other.

Concurrent programming is a programming paradigm that allows multiple tasks to be executed simultaneously, sharing resources and interacting with each other.

It is a way of writing code that allows multiple tasks to be executed at the same time, sharing resources and interacting with each other.

Concurrent programming is a programming paradigm that allows multiple tasks to be executed simultaneously, sharing resources and interacting with each other.

It is a way of writing code that allows multiple tasks to be executed at the same time, sharing resources and interacting with each other.

WHAT IS CONCURRENCY

“

Concurrency is not parallelism

”

Robert Pike

WHAT IS CONCURRENCY

“

*Concurrency is the composition of **independently** executing processes, while parallelism is the **simultaneous** execution of (possibly related) computations.*

”

Robert Pike

WHAT IS CONCURRENCY

“

*Concurrency is about **dealing** with lots of things at once.*

*Parallelism is about **doing** lots of things at once.*

”

Robert Pike

WHAT IS CONCURRENCY

☐ Preserve log ☐ Disable cache No throttling						
press ☐ Invert ☒ Hide data URLs All Fetch/XHR JS CSS Img Media Font Doc WS Wasm Manifest Other ☐ Has blocked cookies ☐ Blocked Requests ☐ 3rd-party req						
Name	Status	Type	Initiator	Size	Time	Waterfall
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	1.44 s	
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	1.45 s	
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	1.41 s	
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	185 ms	
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	176 ms	
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	204 ms	
☐ ?press=hp-dfe	200	fetch	<u>VM18:1</u>	2.8 kB	205 ms	

GOROUTINE

Lightweight thread (few KB)

Run in the same address space

Automatically multiplexed into OS threads

Grow and shrink based on needs

RACE CONDITION

```
i := 0  
for j := 0; j < 3; j++ {  
  i++  
}
```

G1

read i (0)

i++ (1)

read i (1)

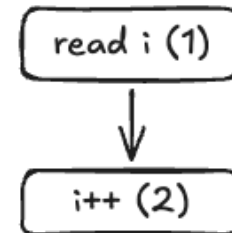
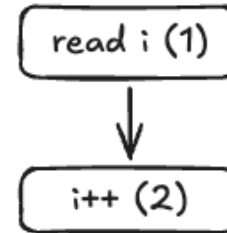
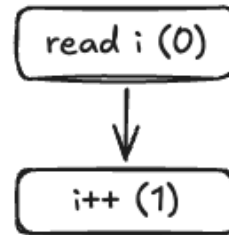
i++ (2)

read i (2)

i++ (3)

RACE CONDITION

```
i := 0  
for j := 0; j < 3; j++ {  
  go func() {  
    i++  
  }()  
}
```



MUTEX



Mutex

```
var m sync.Mutex
```

```
m.Lock() // acquire lock
```

```
m.Unlock() // release lock
```

WAIT GROUP



Wait Group

```
var wg sync.WaitGroup

// add 1 task to the counter
wg.Add(1)

// signal a task completed, -1 to the counter
wg.Done()

// block until counter reaches zero
wg.Wait()

// syntactic sugar for wg.Add(1) + defer wg.Done() (Go 1.25+)
wg.Go(func() {})
```

DEADLOCK

A set of goroutines are **blocked** because they are waiting for each other and none can proceed.



COMMUNICATION

COMMUNICATION

“

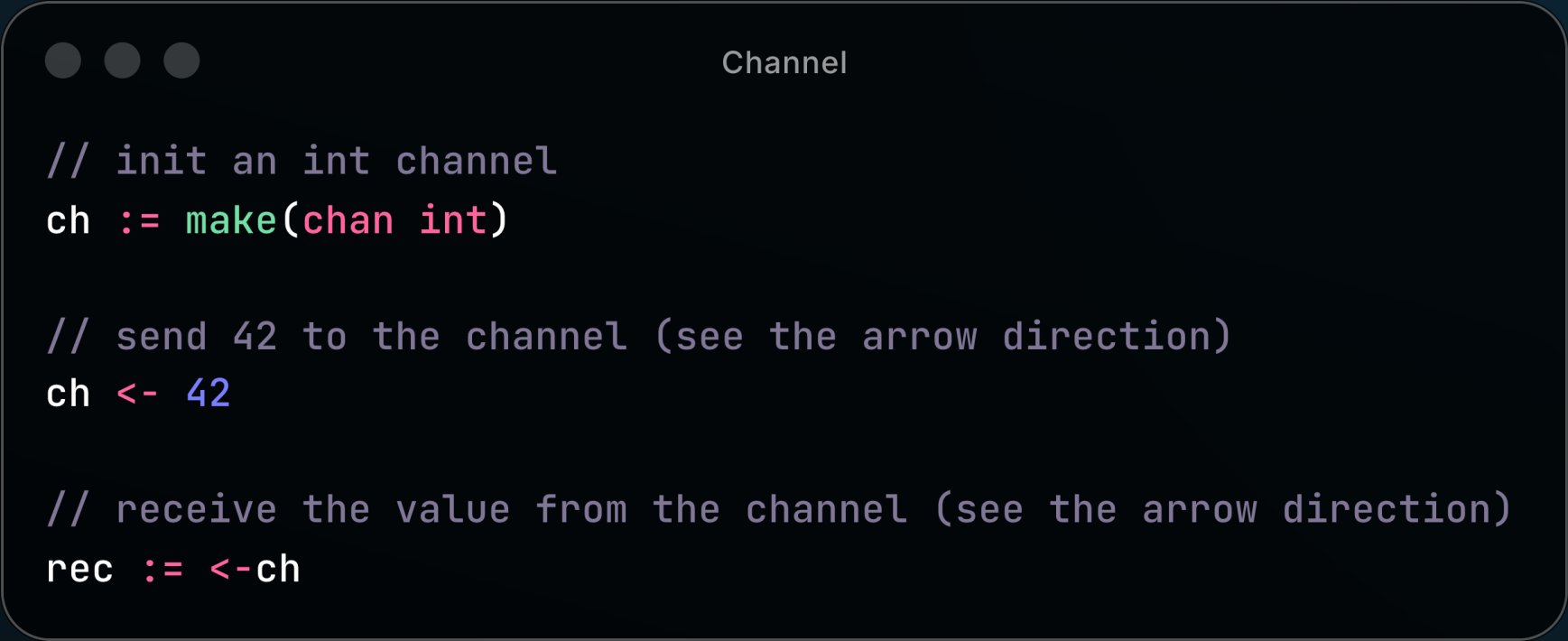
*Don't communicate by **sharing** memory;
share memory by **communicating**.*

”

Robert Pike

CHANNEL

Pipes that connect goroutines — typed and concurrent-safe



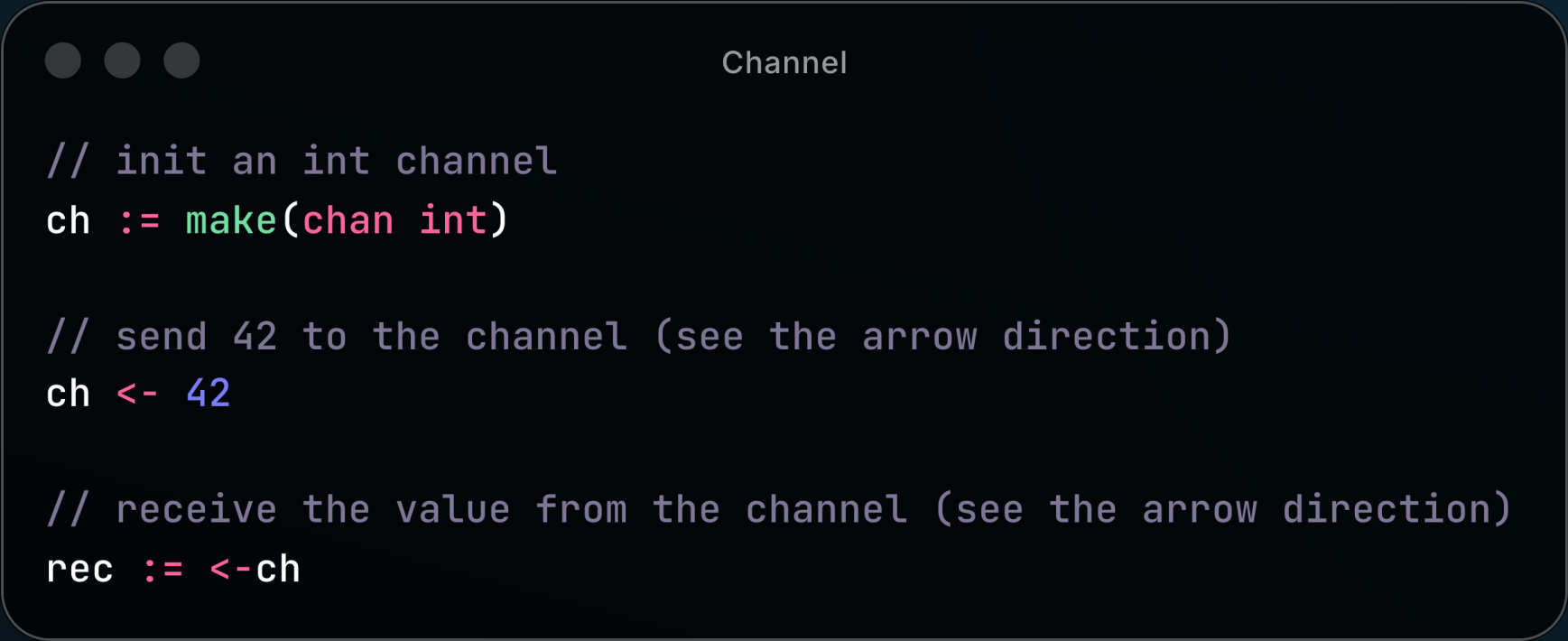
```
// init an int channel
ch := make(chan int)

// send 42 to the channel (see the arrow direction)
ch <- 42

// receive the value from the channel (see the arrow direction)
rec := <-ch
```

CHANNEL

Pipes that connect goroutines — typed and concurrent-safe



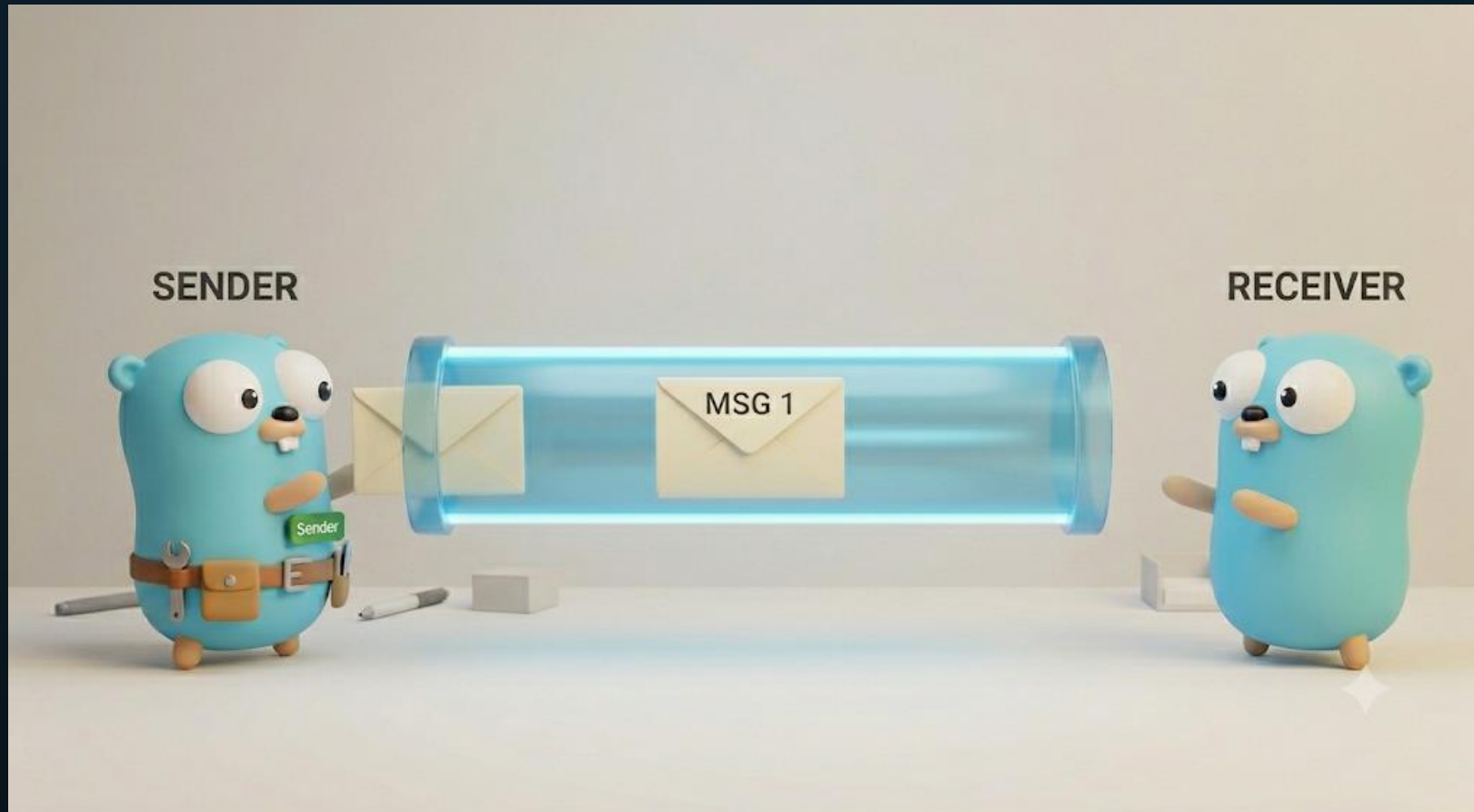
```
// init an int channel
ch := make(chan int)

// send 42 to the channel (see the arrow direction)
ch <- 42

// receive the value from the channel (see the arrow direction)
rec := <-ch
```

Spoiler alert: this code doesn't work (we'll see why later)

CHANNEL



CHANNEL - SYNC

Sender blocks until a receiver is ready

Receiver blocks until a sender is ready

CHANNEL - SYNC

Sender blocks until a receiver is ready

Receiver blocks until a sender is ready

What **ready** means?

UNBUFFERED CHANNEL

Zero capacity, perfect sync

Sender blocks until someone receives

Receiver blocks until someone sends

BUFFERED CHANNEL

Capacity > 0

Sender blocks only when buffer is **full**

Receiver blocks only when buffer is **empty**

CLOSE A CHANNEL

No more messages will be sent through

Usually closed by the sender

Panic if send to a closed channel

Zero value if receive from a closed and empty channel

CLOSE A CHANNEL

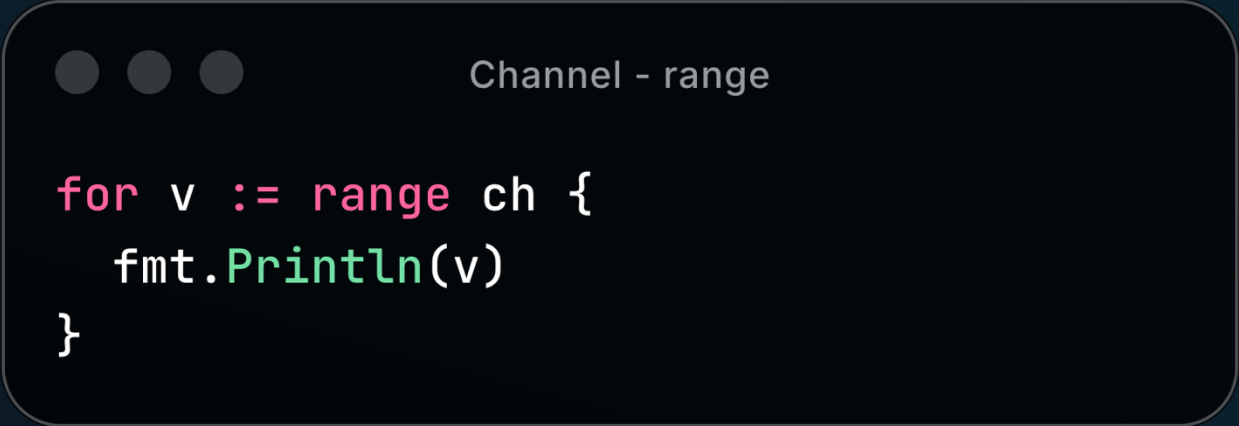


Channel - close

```
close(ch)
```

```
// ok is false if channel is closed and empty  
v, ok := <-ch
```

READ VALUES FROM CHANNEL



```
Channel - range

for v := range ch {
    fmt.Println(v)
}
```

Idiomatic way to loop over a channel

No need to check if channel has been closed

GOROUTINE LEAK

A goroutine that can't terminate, it remains blocked in memory **indefinitely**.

Long-running executions (e.g. servers) are affected by leak.

All goroutines are **terminated** when the main exits.

CHANNEL - PITFALLS

A send to a nil channel **blocks forever**

A receive from a nil channel **blocks forever**

A send to a closed channel **panics**

A receive from an empty and closed channel returns the **zero value**

Closing a nil channel **panics**

CHANNEL – RULES OF THUMB

Always initialize channels before use

Always close channels from the sender side

Close a channel only when you're sure **no goroutine will send** to it anymore

Use **range** to consume a channel — don't check close manually

Prefer unbuffered channels — use buffered only when you have a reason

WORKER POOL



WORKER POOL

1. Init input and output channels
2. Declare WaitGroups for workers and output goroutines
3. Call output goroutine
4. Call workers goroutines
5. Feed input channel

WORKER POOL

6. Close input channel
7. Wait for workers goroutines to end
8. Close output channel
9. Wait for output goroutine to end

SELECT

Listen to multiple channels **simultaneously** and executes the first case that is ready

If multiple cases are ready, one is chosen at **random**

If no case is ready, it **blocks** - unless there is a **default**

SELECT



Select - example

```
ch1 := make(chan string, 1)
ch2 := make(chan string, 1)
// ...
// blocks until one of the channels is ready
// if both are ready, one is chosen at random
select {
    case <-ch1:
        fmt.Println("ch1 wins!")
    case <-ch2:
        fmt.Println("ch2 wins!")
}
```

SELECT

```
● ● ● Select with default - example

ch1 := make(chan string, 1)
ch2 := make(chan string, 1)

// ...

select {
    case <-ch1:
        fmt.Println("ch1 wins!")
    case <-ch2:
        fmt.Println("ch2 wins!")
    default:
        fmt.Println("nobody wins :(")
}
```

SELECT – USE CASES

First Response Wins – multiple goroutines race to complete the same task, the first result wins.

Timeout – wait for a response within a given time limit, cancel if it takes too long.

RECAP

Use Goroutines to run code **concurrently**

Use WaitGroups to know when goroutines are done

Use Channels for **communication**

Close Channels from the sender side

Orchestrate them properly

THANKS



From Race Conditions to Worker Pools: Mastering Go
Concurrency



salvatorecaputi



torejx